Topic Modeling Fundamentals & BERTopic Demo

Arnaldo Santoro, PhD

✉ arnoud.data@gmail.com

arnaldo.santoro@unive.it



Università
Ca'Foscari
Venezia

Summary

Topic Modeling

1. Topic Modeling components
2. BERTopic
3. Evaluation

Topic Modeling

Università
Ca'Foscari
Venezia

Arnaldo Santoro, PhD Student

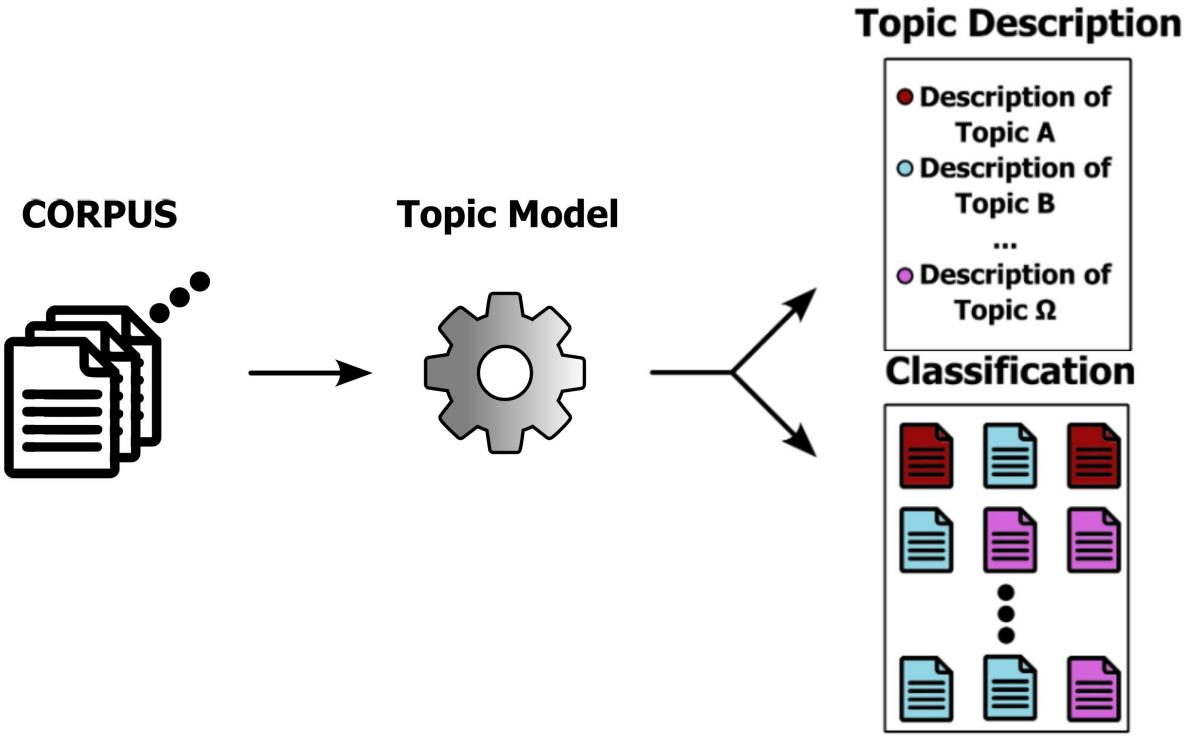
Document Corpus

CORPUS



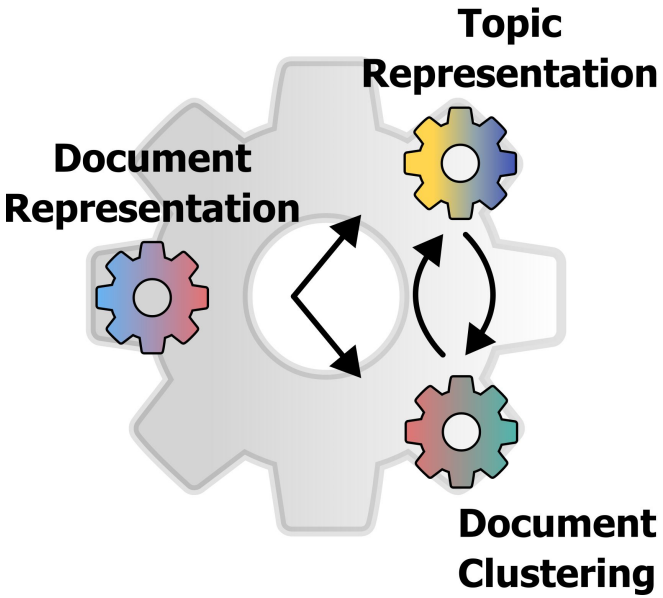
- E.g.: tweets, comments, threads, reviews, abstracts, messages, etc…
- MANY documents
- Limited prior knowledge of contents
- Probably messy

A Model of Topic Modeling



Inside a

Topic Model



Topic Models

LDA, BERTopic, FASTopic, LLM

- Latent Dirichlet Analysis (LDA): long documents, statistical bases, mixed topic assignment, CPU
- BERTopic: short texts, exploratory analysis,unknown topic count, interpretable,

Topic Models: LDA

- **LDA**: Latent Dirichlet Allocation
- **BERTopic**: Bidirectional Encoder Representations from Transformers + Topic
- **FASTopic**: Fast Adaptive Stable Transferable Topic model
- **LLM-based**: Large Language Model - based

When to use?

- 500+ documents
- CPU only
- Long clean same-language documents
- “Known” number of topics
- statistical inference
- soft topic assignment

Topic Models: BERTopic

- **LDA**: Latent Dirichlet Allocation
- **BERTopic**: Bidirectional Encoder Representations from Transformers + Topic
- **FASTopic**: Fast Adaptive Stable Transferable Topic model
- **LLM-based**: Large Language Model - based

When to use?

- 500/500.000 documents
- GPU
- short noisy documents
- unknown topic number
- flexible

Topic Models: FASTopic

- **LDA**: Latent Dirichlet Allocation
- **BERTopic**: Bidirectional Encoder Representations from Transformers + Topic
- **FASTopic**: Fast Adaptive Stable Transferable Topic model
- **LLM-based**: Large Language Model - based

When to use?

- 500+ documents
- GPU
- Long not noisy documents
- known topic number
- soft topic assignment

Topic Models: LLM

- **LDA**: Latent Dirichlet Allocation
- **BERTopic**: Bidirectional Encoder Representations from Transformers + Topic
- **FASTopic**: Fast Adaptive Stable Transferable Topic model
- **LLM-based**: Large Language Model - based

When to use?

- less than 500 documents
- can pay tokens
- possibly not noisy
- nuanced topics
- flexible

BERTopic

Why BERTopic?

- Practical, flexible
- Useful baseline
- Employs many concepts

What is it lacking?

- Research is going elsewhere
- Lacks stability
- Some slow sections

BERTopic

"BERTopic is a topic modeling technique that leverages 🤗 transformers and c-TF-IDF to create *dense clusters* allowing for easily *interpretable topics* whilst keeping *important words* in the topic descriptions."

Maarten Grootendorst

Thank you

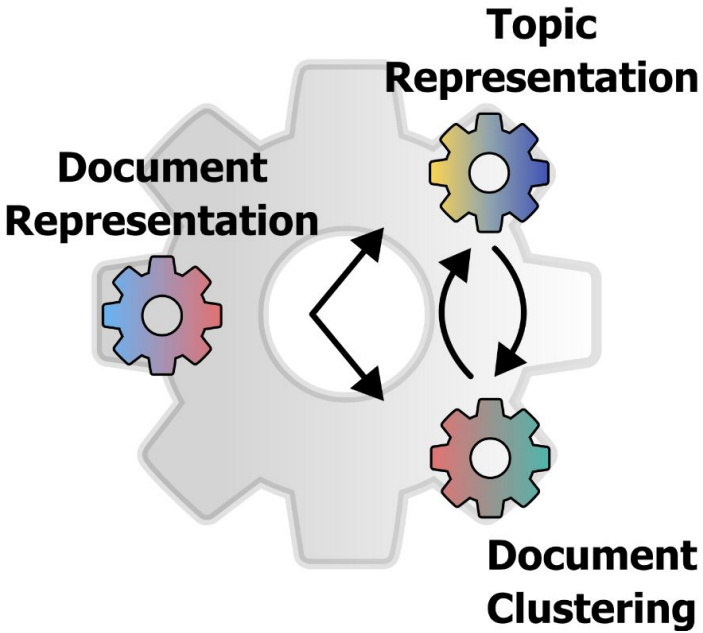


BERTopic

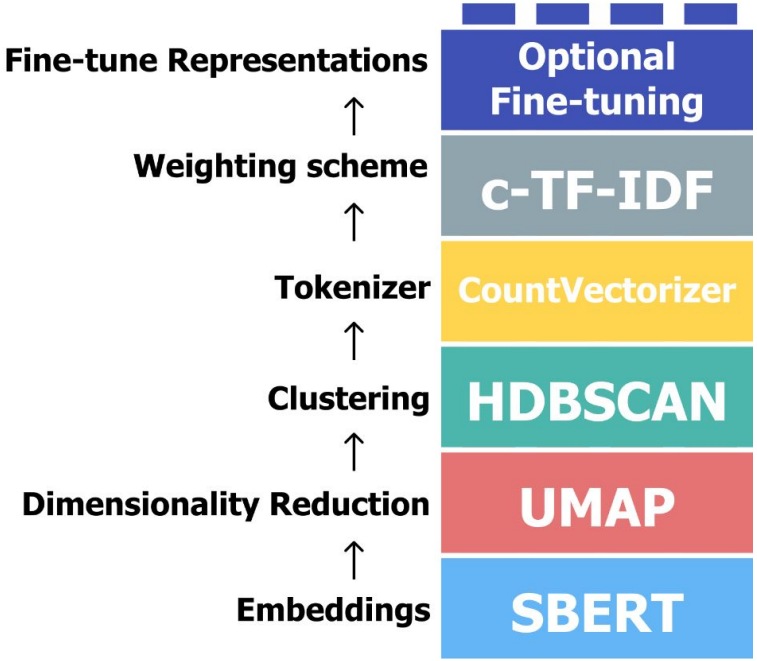
"BERTopic is a topic modeling technique that leverages 🤗 transformers and c-TF-IDF to create *dense clusters* allowing for easily *interpretable topics* whilst keeping *important words* in the topic descriptions."

Maarten Grootendorst

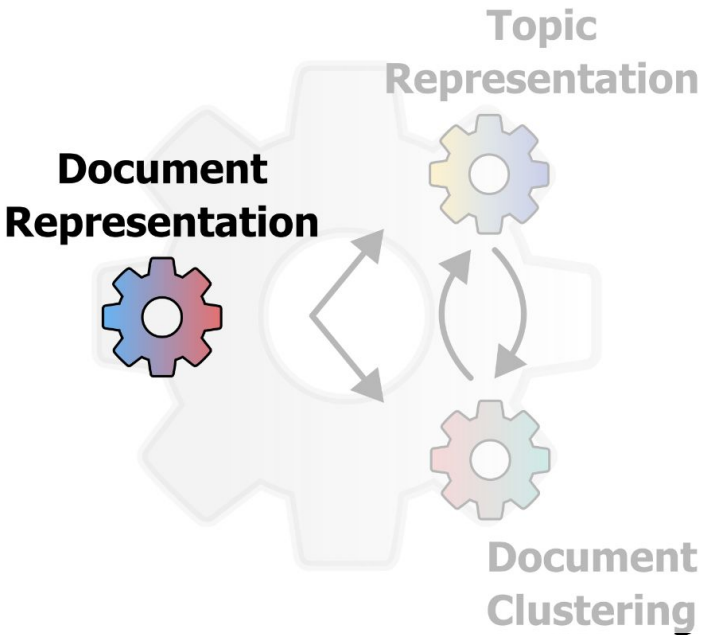
Topic Model



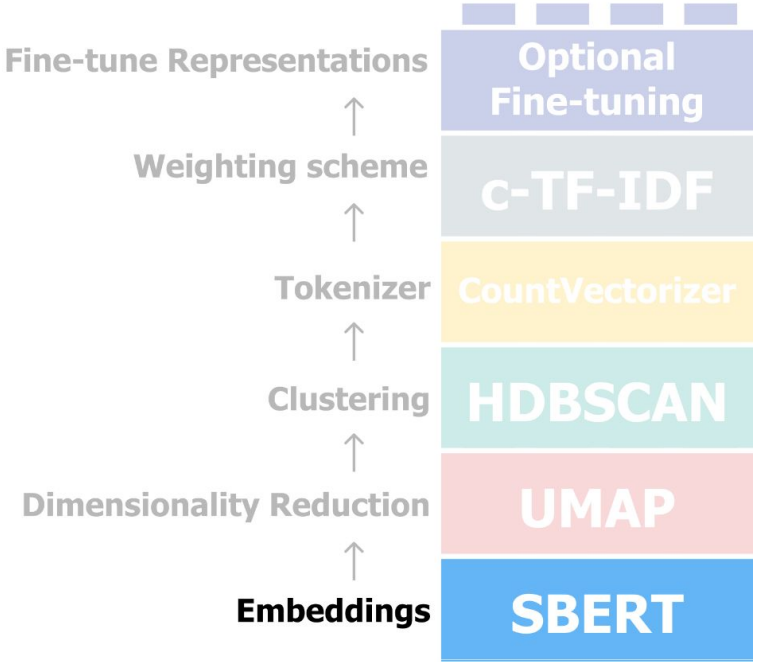
BERTopic



Topic Model



BERTopic

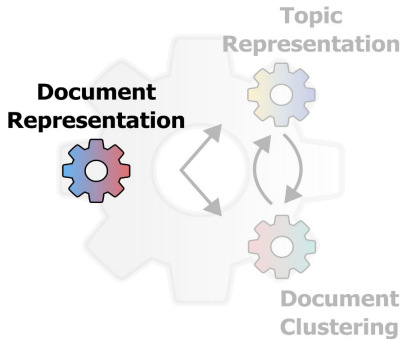


Document representation?

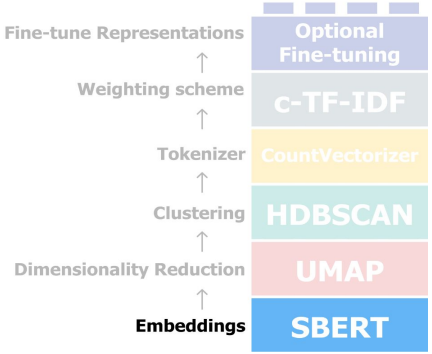
Two big families

- **Term (word) based**
  - bag of words
  - TF-IDF
- **Embedding based**
  - word2vec
  - BERT
  - LLMs

Topic Model



BERTopic



Term based

doc1 = “Parole parole parole, parole soltanto parole, parole d’amor.”

doc2 = “Quanto è bello far l’amore da Trieste in giù.”

bow1 = {“Parole”:1, “parole”:3,“parole,”:2, “soltanto”:1, “d’amor.”:1}

bow2 = {“Quanto”:1, “è”:1, “bello”:1 , “far”:1, “l’amore”:1, “da”:1, “Trieste”:1, “in”:1, “giù.”:1}

Term based

doc1 = “Parole parole parole, parole soltanto parole, parole d’amor.”

doc2 = “Quanto è bello far l’amore da Trieste in giù.”

bow1 = {“amor”:1, “d”:1, “parole”:6, “soltanto”:1}

bow2 = {“amore”:1, “bello”:1, “da”:1, “e”:1, “far”:1, “giu”:1, “in”:1, “l”:1, “trieste”:1, “quanto”:1}

Term based

doc1 = “Parole parole parole, parole soltanto parole, parole d’amor.”

doc2 = “Quanto è bello far l’amore da Trieste in giù.”

bow1 = {“amore”: 1, “parole”: 6, “soltanto”: 1}

bow2 = {“amore”:1, “bello”:1, “fare”:1, “sotto”:1, “Trieste”:1, “quanto”:1}

Bag of Words (BoW)

Documents represented by the frequency of its terms.

Simple, interpretable, requires ad hoc preprocessing

Loss of syntactic and semantic information.

BoW: Limits

n-grams: “Financial Times”, “New York”.

Homographies: “axes”, “jam”, “lie”, “wave”.

Enantiosemy: “dust ”, “ravel”, “overlook”, “sanction”.

Terms’ position:

- “John said Mary kissed Peter” + “only”

Stemming and lemmatization can modify terms semantics:

- “jumping” -> “jump” <- “jumper”

Term based recap

Bag of Words

Advantages:

- Simple
- Interpretable
- Explainable

Limits:

- Information loss
- Requires ad hoc preprocessing
- Takes space

Embeddings

Store the meanings

Create a vector where dimension 1 is for concepts, e.g. female/male.

“Close” vectors should represent similar words.

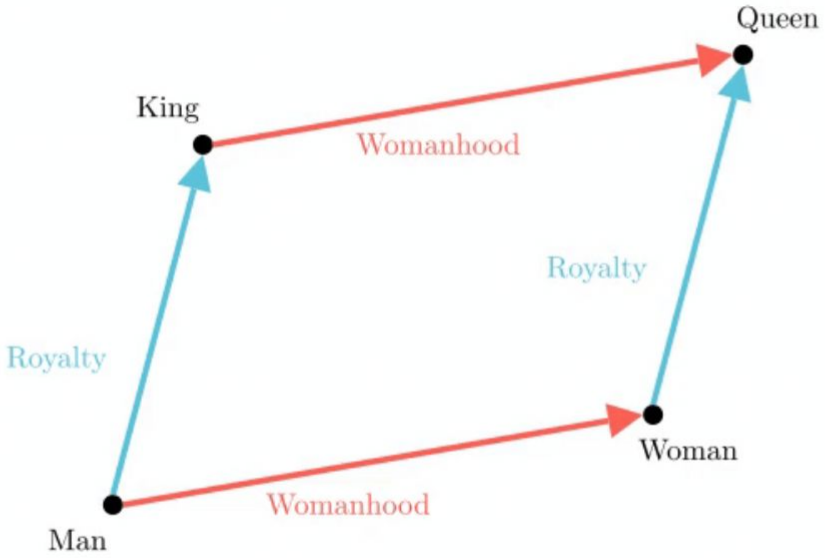
How?

Train an AI to learns relationships between words.

Embeddings

1) Many precomputed models.
2) Keep context and meaning of words.
3) Spare space on disk/RAM.
4) Allow “mathematical” manipulation.

Embedding Math



Embedding Math

$$\overrightarrow{KING} - \overrightarrow{MAN} + \overrightarrow{WOMAN} = \overrightarrow{QUEEN}$$

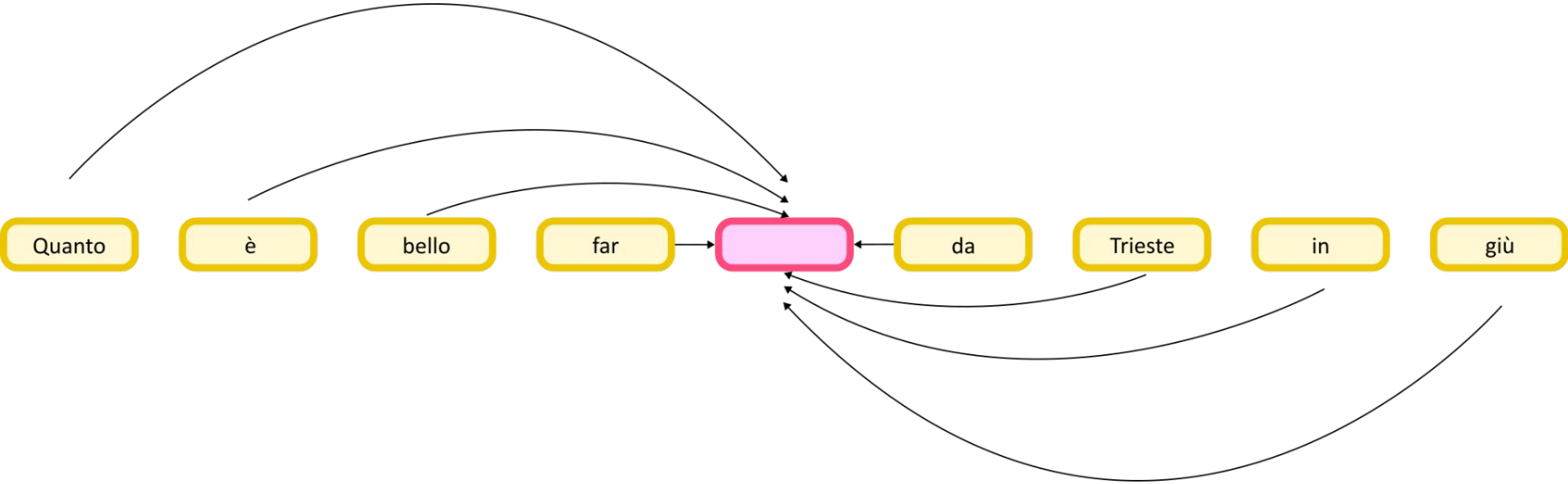
$$\overrightarrow{ROME} - \overrightarrow{ITALY} + \overrightarrow{SPAIN} = \overrightarrow{MADRID}$$

$$\overrightarrow{FASCISM} - \overrightarrow{ITALY} + \overrightarrow{GERMANY} = \overrightarrow{NAZISM}$$

$$\overrightarrow{DOCTOR} - \overrightarrow{MAN} + \overrightarrow{WOMAN} = \overrightarrow{NURSE}\ *$$

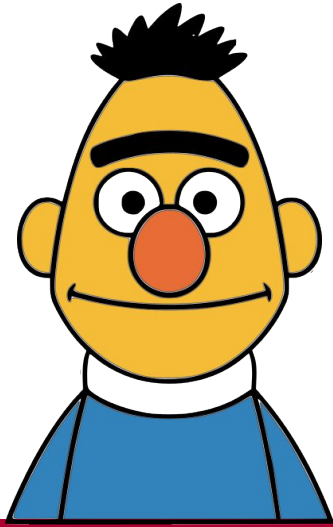
* embeddings may learn relationships from FLAWED training texts.

Learning Embeddings



BERT

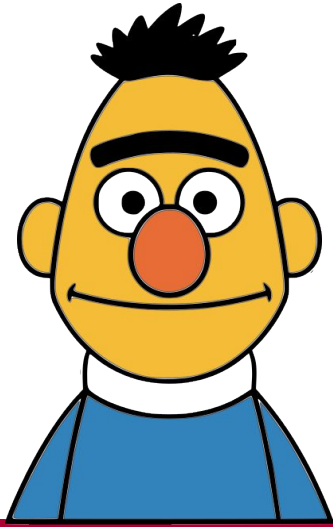
- **BERT (2018)**
- **B**idirectional **E**ncoder **R**epresentations from **T**ransformers
- Bidirectional: predict words reading text before and after it
- Transformer: neural network architecture to process sequential data
- Learns relationship between words in the corpus



SBERT

- **Sentence BERT (2019)**
- From words to sentences
- Uses (mean) pooling on word embeddings
- Limit to document length (~500 tokens≈ 300 words)

$$\text{Sentence} \xrightarrow{\text{BERT}} \begin{pmatrix} \textbf{Embedding}_{[\text{CLS}]} \\ \textbf{Embedding}_{word_1} \\ \textbf{Embedding}_{word_2} \\ \vdots \\ \textbf{Embedding}_{word_T} \end{pmatrix}$$



Embeddings recap

SBERT

Advantages:

- No text preprocessing
- Keeps context
- Less disk space required

Limits:

- Uninterpretable
- Text length limits
- Training corpus Bias

After the embedding

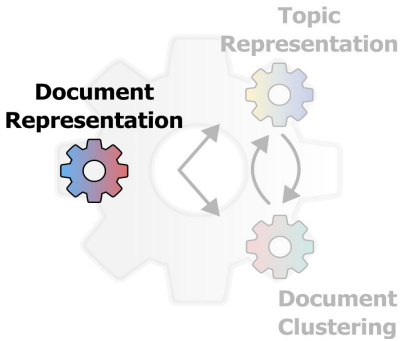
We represented each document in our corpus as a point in a 300+ dimensional space.

Do we group points together…?

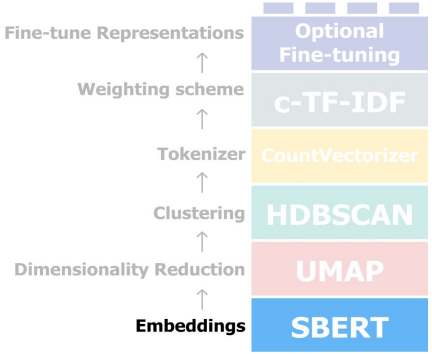
300 dimensions are too many!

Curse of dimensionality!

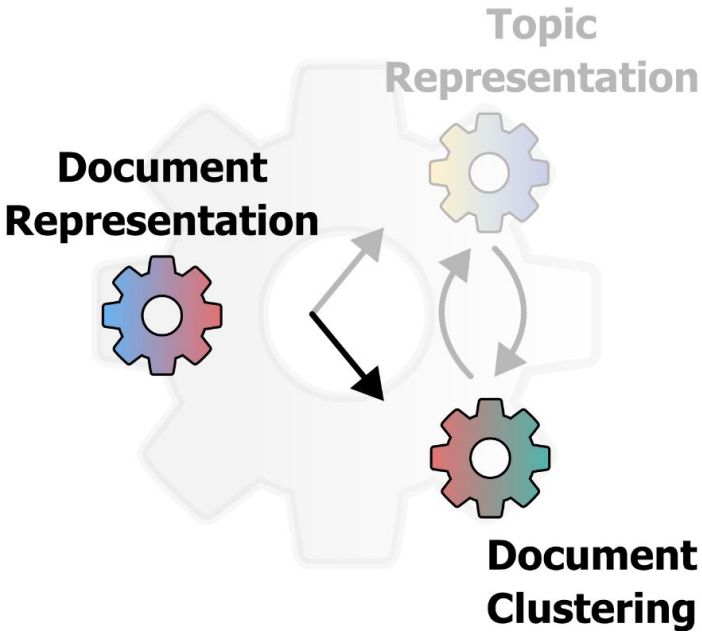
Topic Model



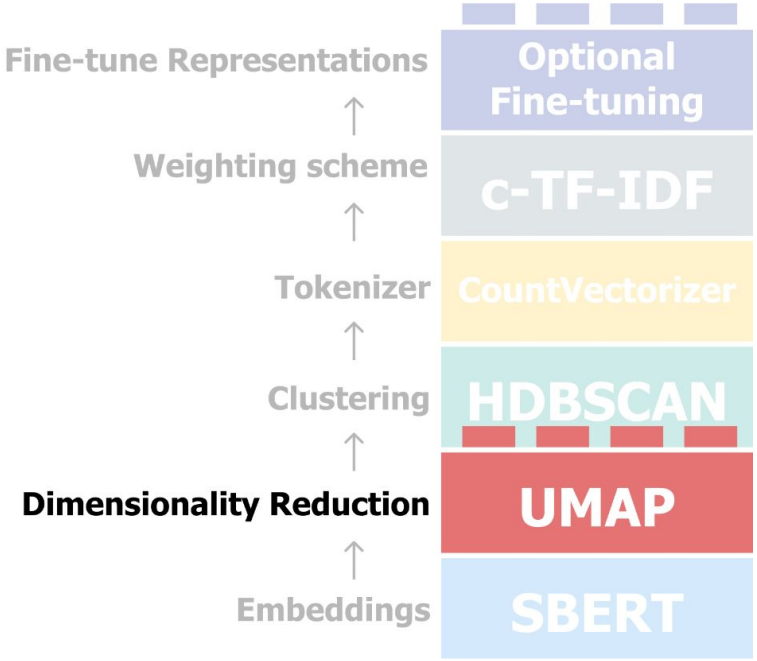
BERTopic



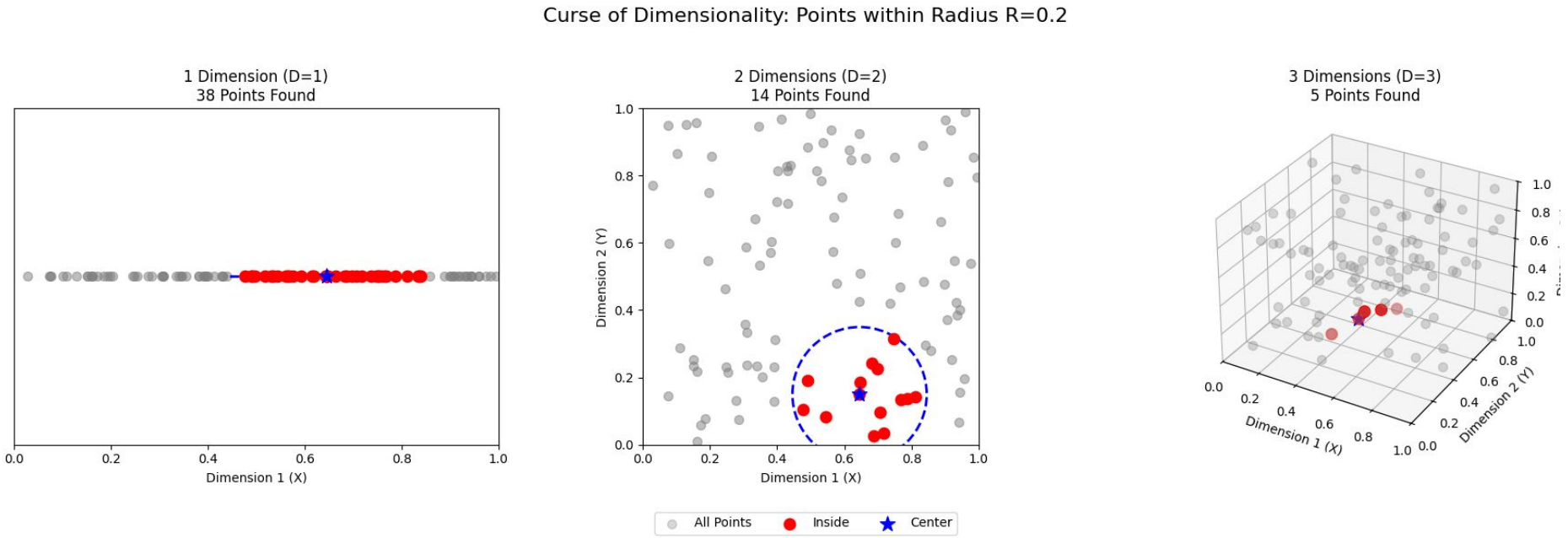
Topic Model



BERTopic



Curse of dimensionality



Dimensionality Reduction

Dimensionality reduction algorithms: PCA, tSNE, **UMAP**, ...

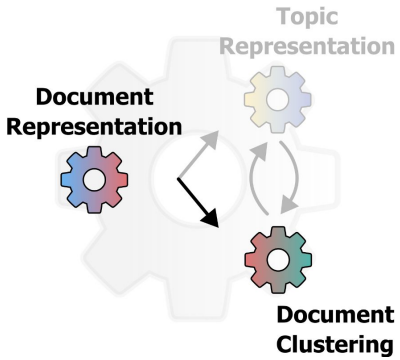
Uniform Manifold Approximation and Projection for dimension reduction - UMAP

Reduce dimensions.

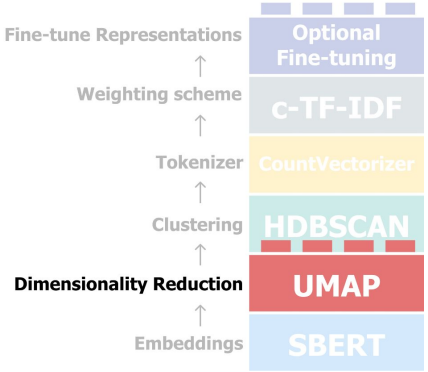
Preserve information.

Let us give a concrete example.

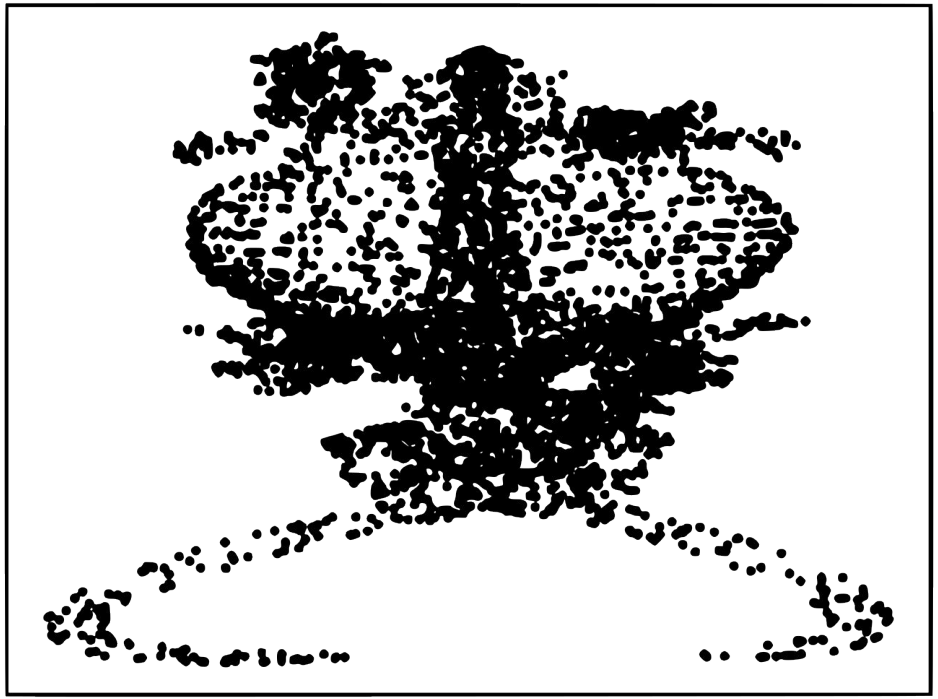
Topic Model



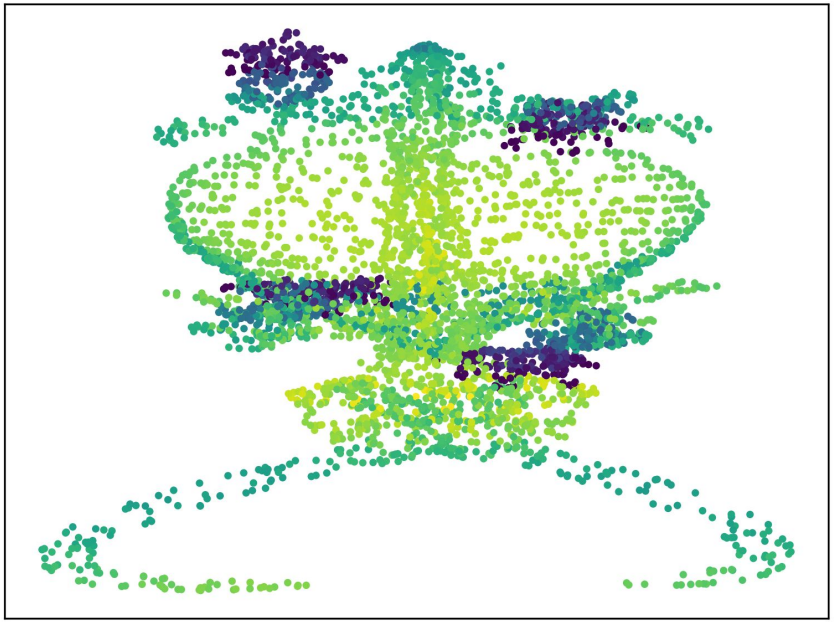
BERTopic

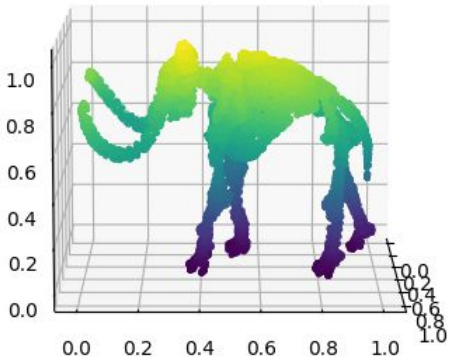


What fossil is this?

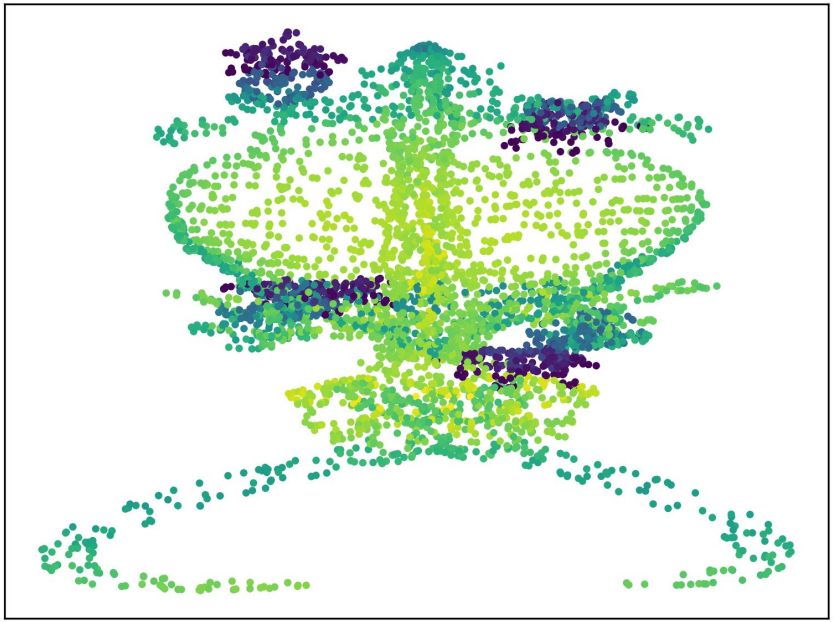


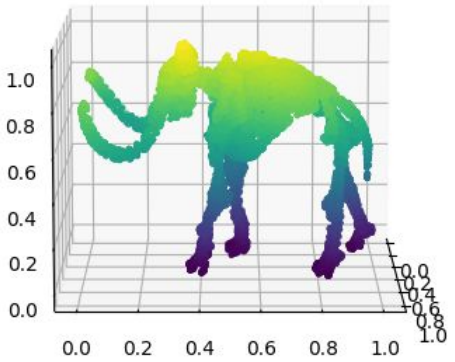
What fossil is this?





Did you guess right?





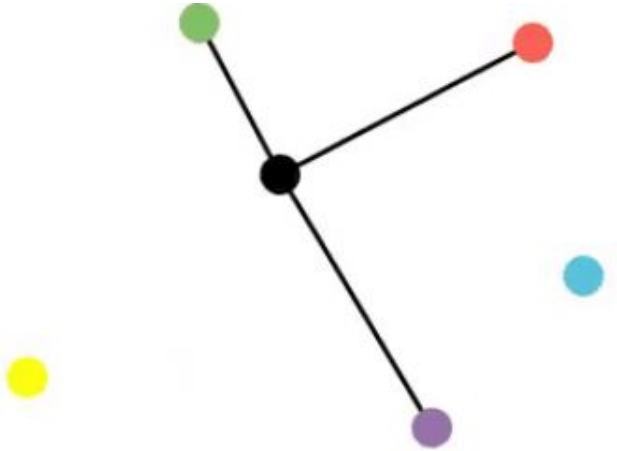
UMAP 1/7

We compute a high dimensional graph and a small dimensional graph.

Then, we refine the small dimensional one to make it similar to the high dimensional one.

How do we compute these graphs ?

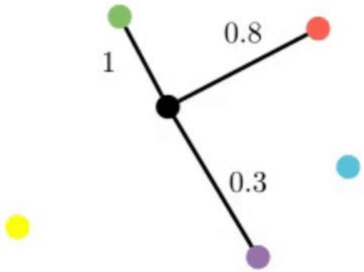
UMAP 2/7



UMAP 3/7

Weight computation

$$v_{i,j} = e^{-\frac{max(0, d(x_i, x_j) - \rho_i)}{\sigma_i}}$$

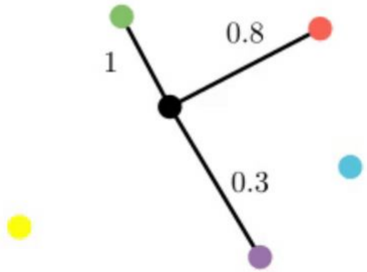


ρ_i = distance to nearest neighbor

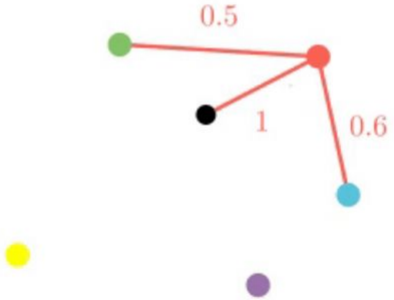
UMAP 4/7

How do we combine these graphs ?

$$w_{ij} = v_{i,j} + v_{j,i} - v_{i,j}.v_{j,i}$$



$$w_{ij} = 1 + 0.8 - 1 \times 0.8$$



UMAP 5/7

Then:

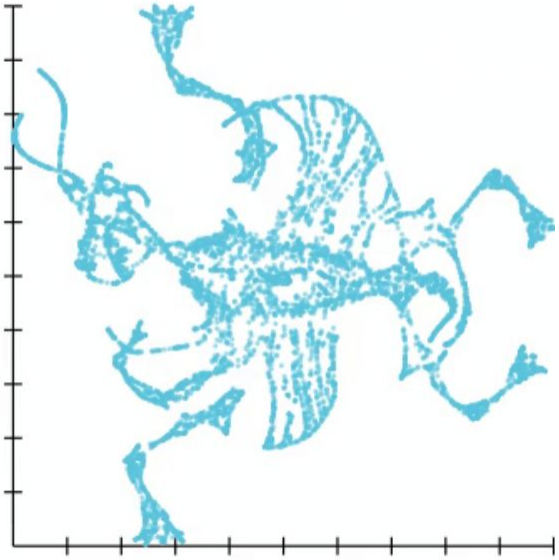
- Create the high and low dimensional representation.
- Compare them, e.g. with cross entropy loss function.
- Minimize distance, e.g. with stochastic gradient descent.
- Change the low representation accordingly

We obtain a projection which tries retains the structure of our data.

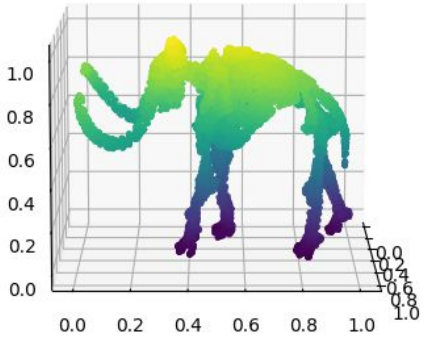
Success depends on K, the number of neighbours we consider (and on the data).

UMAP 6/7

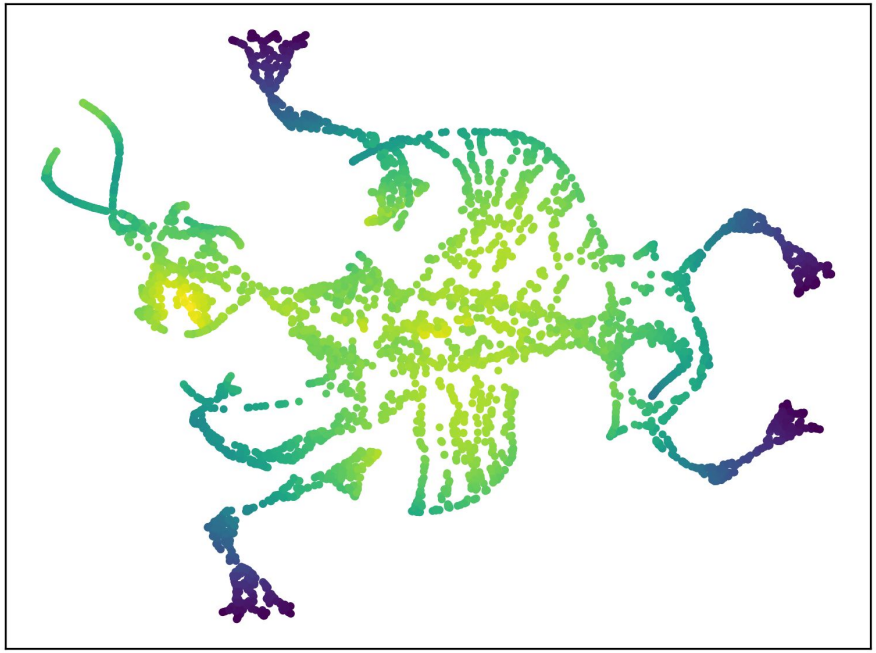
UMAP with 100 neighbors



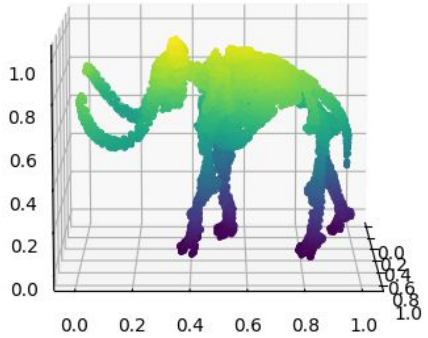
Original 3D data



UMAP 7/7



Original 3D data

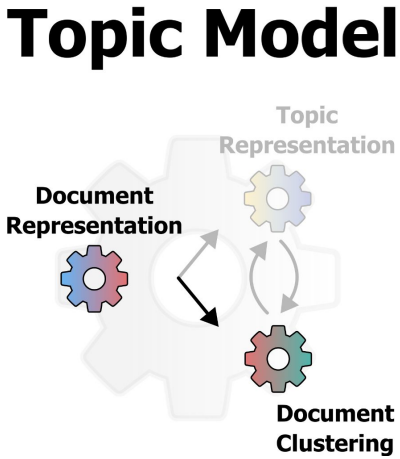


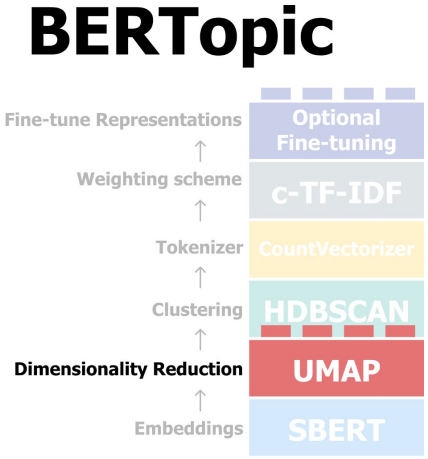
After UMAP

We decimated our 300+ dimensions.

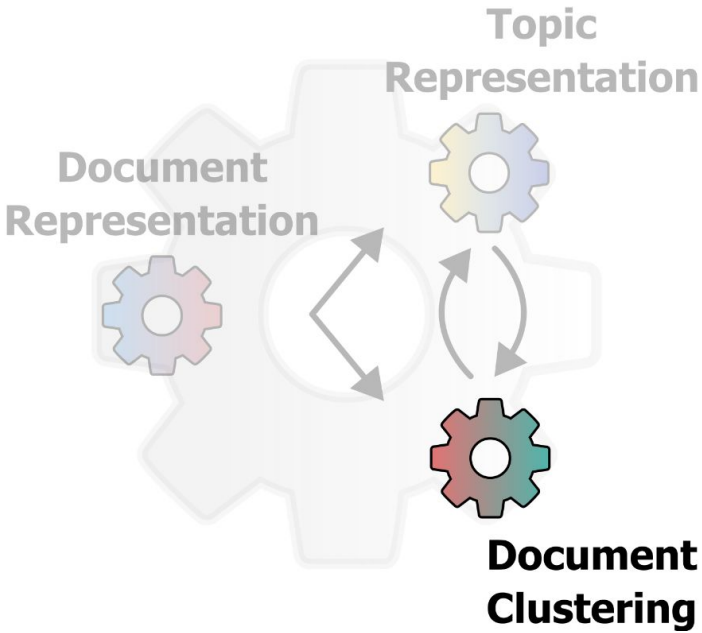
The curse of dimensionality is broken.

Now we group documents together.

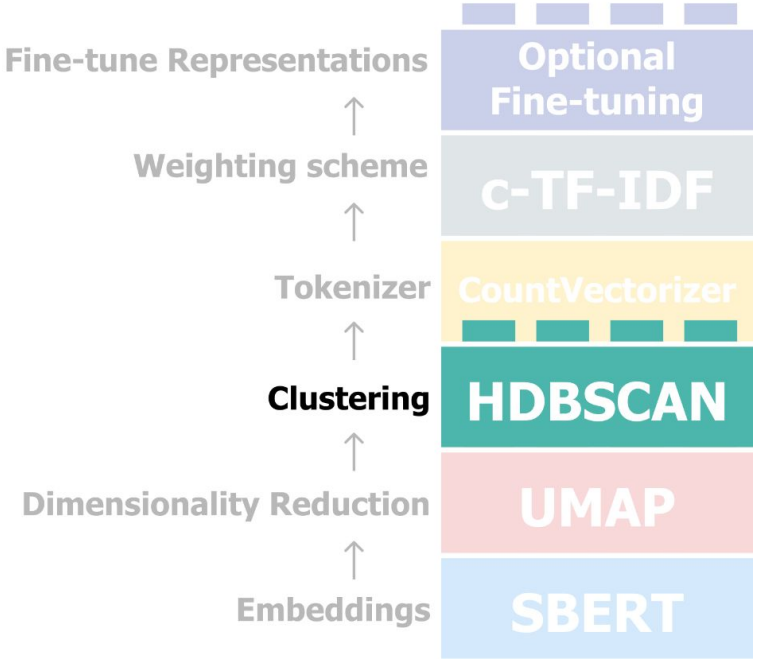




Topic Model



BERTopic



How many clusters are there?



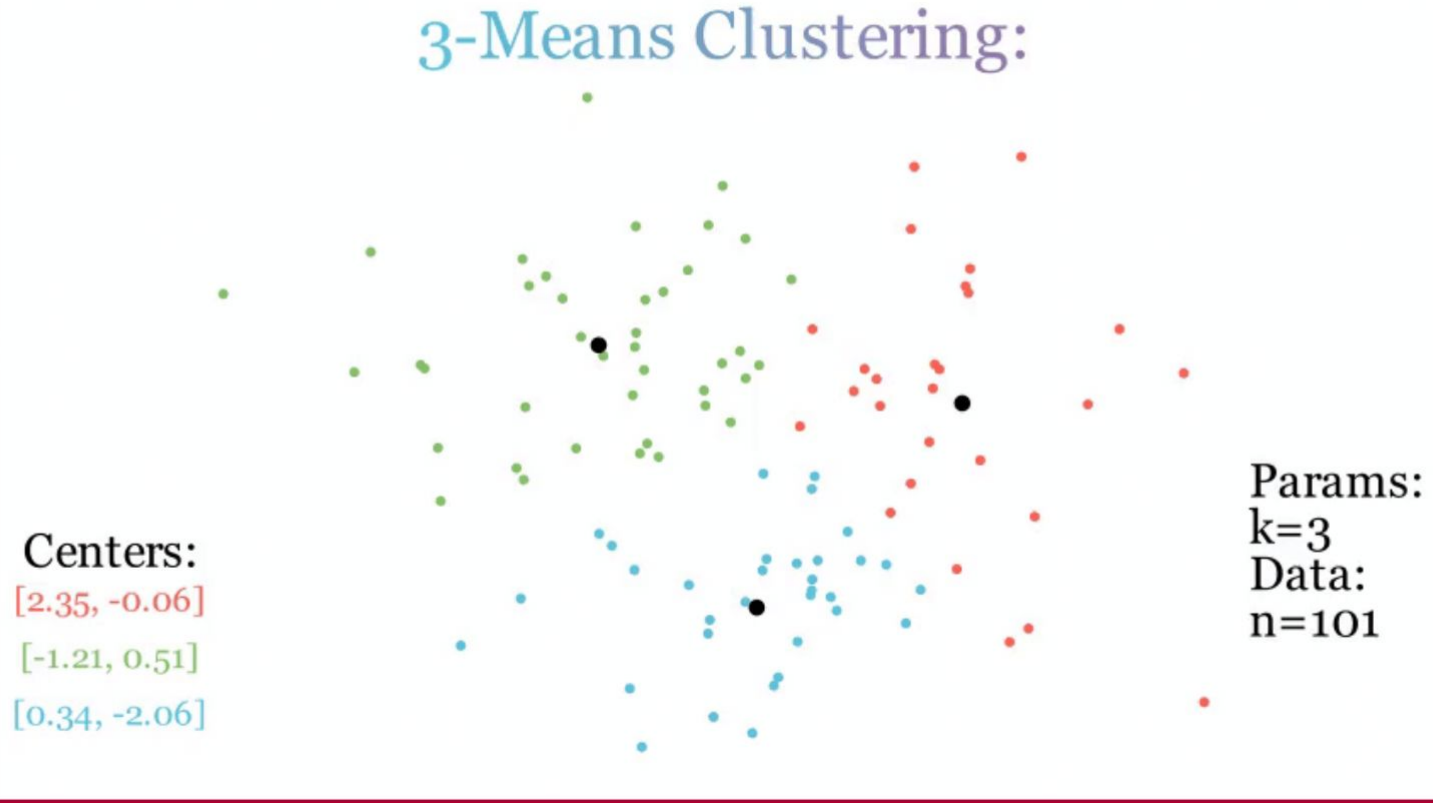
Clustering Methods

Cluster type\Result	Flat	Hierarchical	…
Centroid	K-Means	Hierarchical K-Means	…
Density	DBSCAN	HDBSCAN	…
…	…	…	…

K-Means

- Create K “centers”.
- Until the partition is stable:
  - Assign each point to the closest center.
  - Recompute the center’s location.

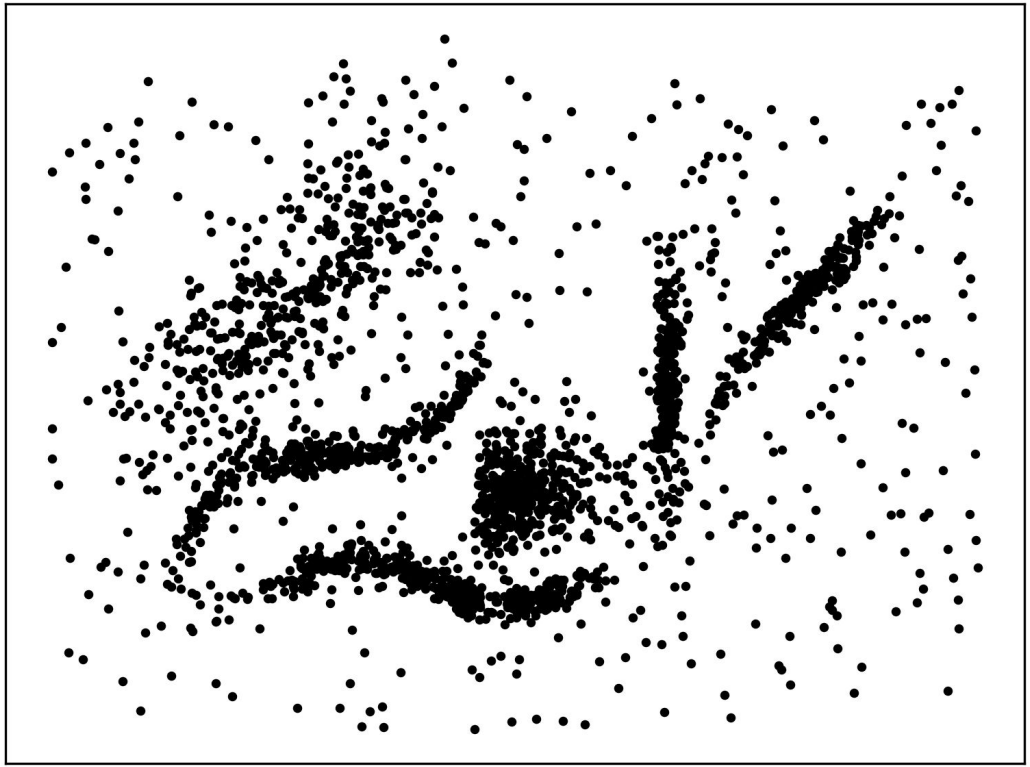
Example: K=3


3-Means Clustering:
Centers:
[2.35, -0.06]
[-1.21, 0.51]
[0.34, -2.06]
Params:
k=3
Data:
n=101

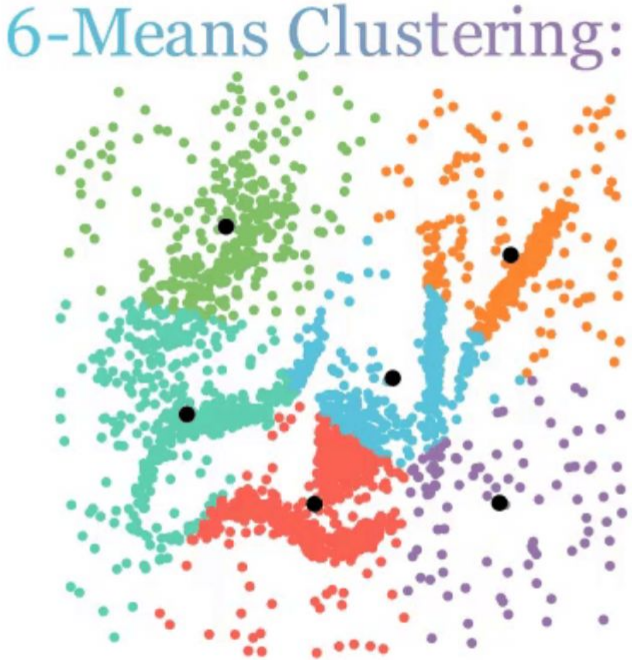
Example: K=6


6-Means Clustering:
Centers:
[-2.81, -0.35]
[2.88, 1.16]
[0.25, -2.27]
[2.3, -1.67]
[1.08, 0.03]
[-0.83, 0.74]
Params:
k=6
Data:
n=101

Data with K=6



K-Means



Centers:
[0.0, -1.04]
[-0.86, 1.65]
[0.77, 0.18]
[1.92, 1.38]
[-1.24, -0.17]
[1.81, -1.03]

Params:
k=6
Data:
n=2310

DBSCAN Algorithm

Density Based Spatial Clustering of Applications with Noise

Pick values for *minClusterSize* and ε:

- Distinguish **core points** (have at least *minClusterSize* points within distance ε) from **isolated points**.
- Connect **core points** between them (create clusters finding connected components).
- Assign **isolated points** to clusters within distance ε, otherwise assign them to *noise*.

DBSCAN

DBSCAN:



Params:
$\varepsilon = 0.2$
Min size = 20
Clusters: 6
Noise: 587

Data:
n=2310

DBSCAN

Advantages:

- Clusters of any form
- Detects noise
- Not slow on average O(n log n)
- Almost deterministic

Limits:

- Two hyperparameters: ε, minPts
- Slow worst case $O(n^2)$
- Handles ~50 dimensions at most
- Fixed cluster density

HDBSCAN

Hierarchical DBSCAN

Advantages:

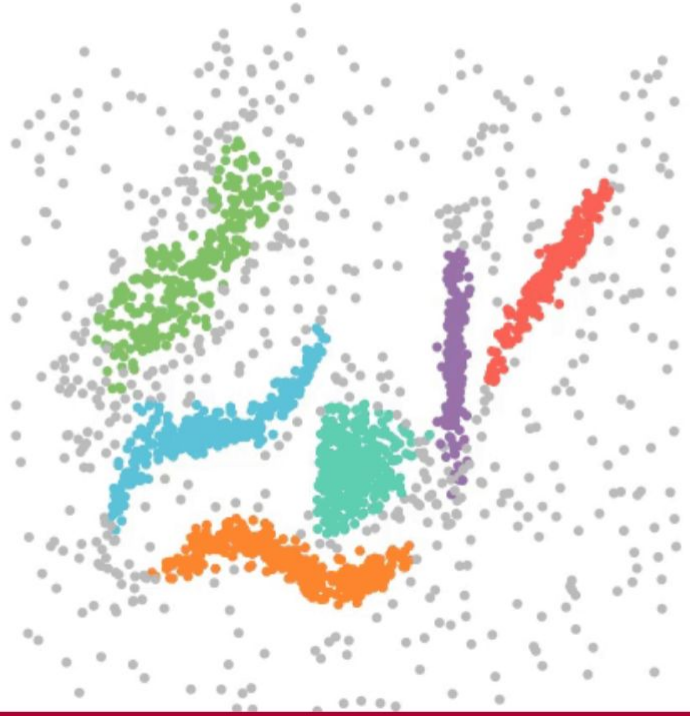
- DBSCAN's
- Clusters may have different densities
- Only one hyperparameter

Limits:

- Slow in the worst case $O(n^2)$
- Handles ~50 dimensions

HDBSCAN

HDBSCAN:



Params:
Min size = 15
Clusters: 6
Noise: 566

Data:
n=2310

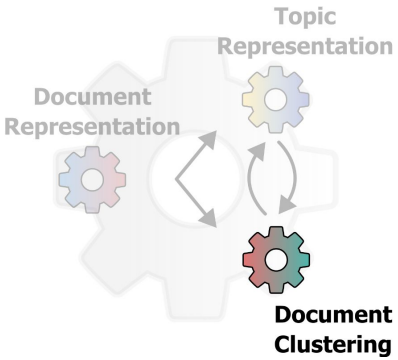
Clustering

Group homogeneous elements together.

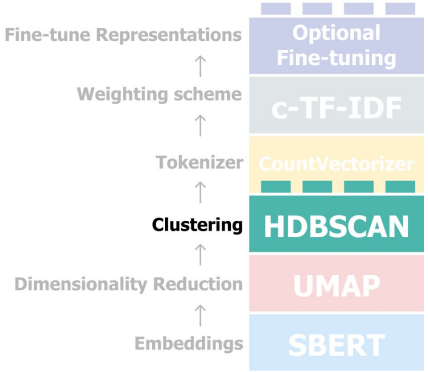
Unsupervised learning: autonomously discover patterns.

There is no **one and only** truth.

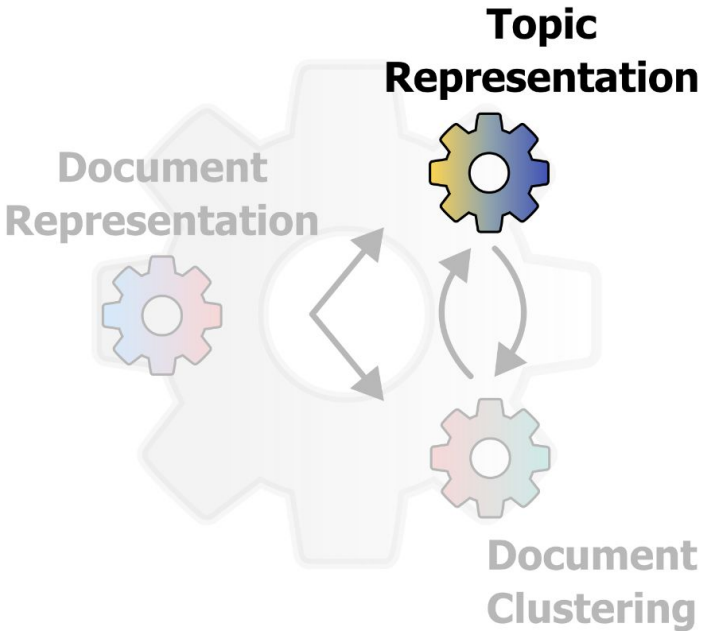
Topic Model



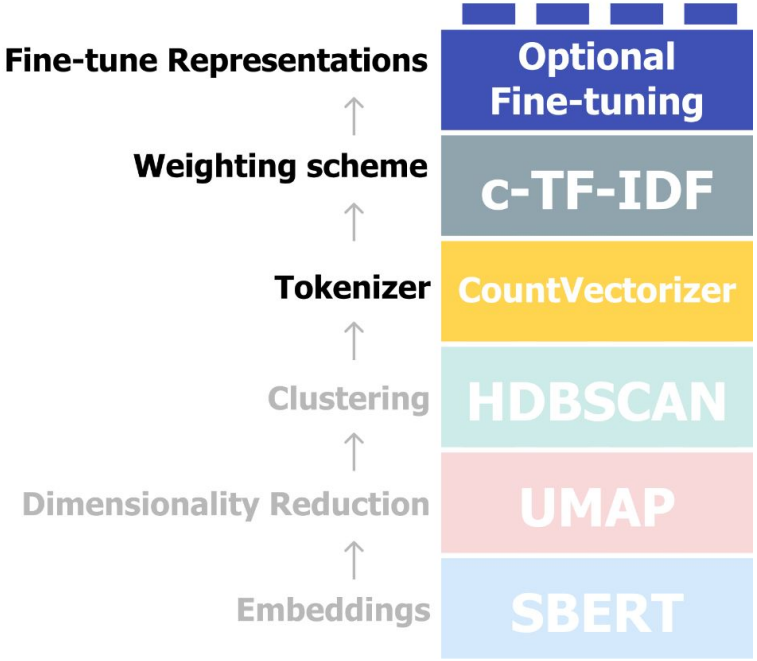
BERTopic



Topic Model



BERTopic



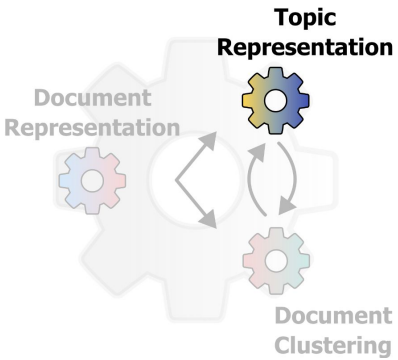
Represent Topics

Embeddings are not human interpretable, but words are.

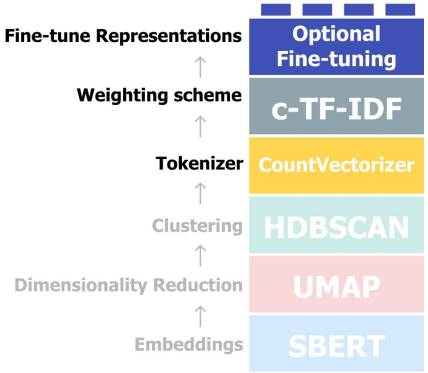
Base topic representation: sequence of “important” words.

…BoW?

Topic Model



BERTopic



BoW?

Is a **frequent** word important?

"necessarily"

And a **rare** one?

"necesaarily"

class Term Frequency - Inverse Document Frequency

c-TF-IDF

Given a term t and a class c, a word's importance is given by:

$$W_{t,c} = f_{t,c} \times log(1 + \frac{A}{f_t})$$

where $f_{t,c}$ is the frequency of t in class c

A is the mean number of words per class,

f_t is the frequency of term t in the whole corpus.

c-TF-IDF measures word importance.

Other Representations

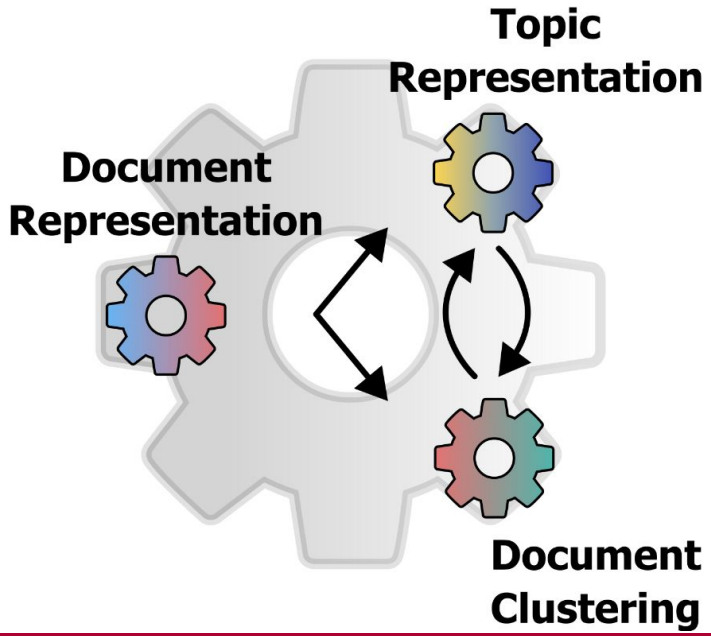
- Marginal Maximum Relevance
- Part Of Speech
- KeyBERT
- **LLMs**



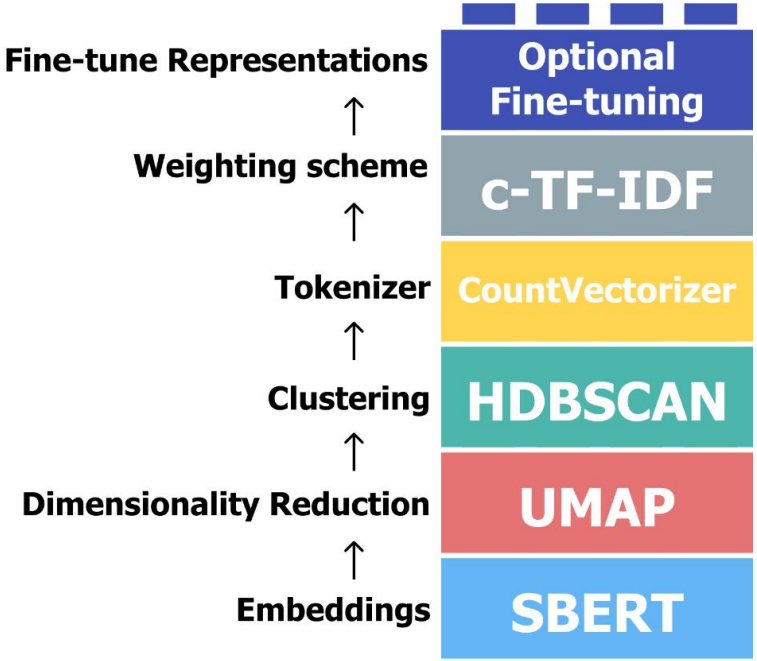
Top 10 LLMs in 2026
https://www.sevensquaretech.com/best-llm-models/

We've topic-modeled. What now?

Topic Model



BERTopic



Model Evaluation

Università
Ca'Foscari
Venezia

Arnaldo Santoro, PhD Student

Model evaluation

We choose:

- Embeddings
- UMAP: #neighbors and #minClusterSize
- HSBSCAN: minimum cluster size, …
- Representation: language, stopwords, …

Did we pick right? Is our model good? Can we do better?

How do we decide?

Model evaluation

Produce multiple models, with multiple parameters

Measure aspects each

Then pick the best

i.e. look at number: number high = good; number low = bad.

Model evaluation

Why?

We need to set different parameters.

Our BERTopic model:

- measure our clustering quality;
- measure our topic representation.

Our Topic model vs other models:

- measure performance of this model vs other models.

Embedding quality

Dimension:

- Big model ⇒ Good model.
- Big model ⇒ Slow model.
- Good model ⇏ Useful model.

High Score in *Performance Sentence Embeddings*.

Dataset used on training.

Supported languages.

https://www.sbert.net/docs/sentence_transformer/pretrained_models.html

SBERT: pretrained models

Model Name	Performance Sentence Embeddings (14 Datasets)	Performance Semantic Search (6 Datasets)	Avg. Performance	Speed	Model Size
all-mpnet-base-v2	69.57	57.02	63.30	2800	420 MB
multi-qa-mpnet-base-dot-v1	66.76	57.60	62.18	2800	420 MB
all-distilroberta-v1	68.73	50.94	59.84	4000	290 MB
all-MiniLM-L12-v2	68.70	50.82	59.76	7500	120 MB
multi-qa-distilbert-cos-v1	65.98	52.83	59.41	4000	250 MB
all-MiniLM-L6-v2	68.06	49.54	58.80	14200	80 MB
multi-qa-MiniLM-L6-cos-v1	64.33	51.83	58.08	14200	80 MB
paraphrase-multilingual-mpnet-base-v2	65.83	41.68	53.75	2500	970 MB
paraphrase-albert-small-v2	64.46	40.04	52.25	5000	43 MB
paraphrase-multilingual-MiniLM-L12-v2	64.25	39.19	51.72	7500	420 MB
paraphrase-MiniLM-L3-v2	62.29	39.19	50.74	19000	61 MB
distiluse-base-multilingual-cased-v1	61.30	29.87	45.59	4000	480 MB
distiluse-base-multilingual-cased-v2	60.18	27.35	43.77	4000	480 MB

BERTopic Default

paraphrase-multilingual-MiniLM-L12-v2	64.25	39.19	51.72	7500	420 MB

paraphrase-multilingual-MiniLM-L12-v2

Base Model:	Teacher: paraphrase-MiniLM-L12-v2; Student: microsoft/Multilingual-MiniLM-L12-H384
Max Sequence Length:	128
Dimensions:	384
Normalized Embeddings:	false
Suitable Score Functions:	cosine-similarity (`util.cos_sim`)
Size:	420 MB
Pooling:	Mean Pooling
Training Data:	Multi-lingual model of paraphrase-multilingual-MiniLM-L12-v2, extended to 50+ languages.
Model Card:	https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2

DB Cluster Relative Validity

Between -1 and 1.

This clustering can be compared to other clustering **on this data**.

It is the weighted average of each Cluster's Relative Validity (CRV)

$$CRV = \frac{Min\ Density\ Separation\ (to\ nearest\ cluster) - Max\ Internal\ Sparseness}{The\ Largest\ between\ Min\ Density\ Separation\ and\ Max\ Internal\ Sparseness}$$

Topic Coherence

Measures topic representation against topic content.

How often the topic's words co-occur in the data.

Requires tokenization. Used to compare any topic model, **as long as it uses keyword representation**.

Topic Diversity

Having many identical topics may produce high coherence.

Topic diversity measures uniqueness and variety of the topics.

Many ways to measure it: terms, embeddings.

Most common: Jaccard Topic Diversity.

|unique topic-words | / | topic-words|

BERTopic recap

"Technique that leverages 🤗 transformers and c-TF-IDF to create *dense clusters* allowing for easily *interpretable topics* whilst keeping *important words* in the topic descriptions."

Advantages:

- scalable
- preserves context
- flexible
- interpretable

Disadvantages:

- requires GPU
- **many** important hyperparameters
- document length

Demo

Bibliography

Embeddings:

- https://arxiv.org/abs/1805.04032
- https://en.wikipedia.org/wiki/Word_embedding

BERTopic:

- BERTopic paper: https://arxiv.org/abs/2203.05794
- BERT: https://aclanthology.org/N19-1423.pdf
- UMAP paper: https://arxiv.org/abs/1802.03426
- HDBSCAN paper: https://joss.theoj.org/papers/10.21105/joss.00205.pdf
- John Healy on HDBSCAN https://www.youtube.com/watch?v=dGsxd67IFiU&t=1628s

Animations:

- https://github.com/ytdeepia/Latent-Space-Visualisation-PCA-t-SNE-UMAP/
- https://github.com/sid-sr/k-means-visualiser

Illustrations:

- Cog by Dave Gandy, https://fortawesome.github.com/Font-Awesome/, License CC sa3.0

Bibliography

Useful sites:

- BERTopic: https://maartengr.github.io/BERTopic/index.html
- SBERT pretrained models: https://www.sbert.net/docs/sentence_transformer/pretrained_models.html
- UMAP: https://umap-learn.readthedocs.io/en/latest/
- HDBSCAN: https://hdbscan.readthedocs.io/en/latest/index.html
- OCTIS Topic Model comparison: https://github.com/MIND-Lab/OCTIS
- FASTopic: https://github.com/bobxwu/FASTopic

Thank you

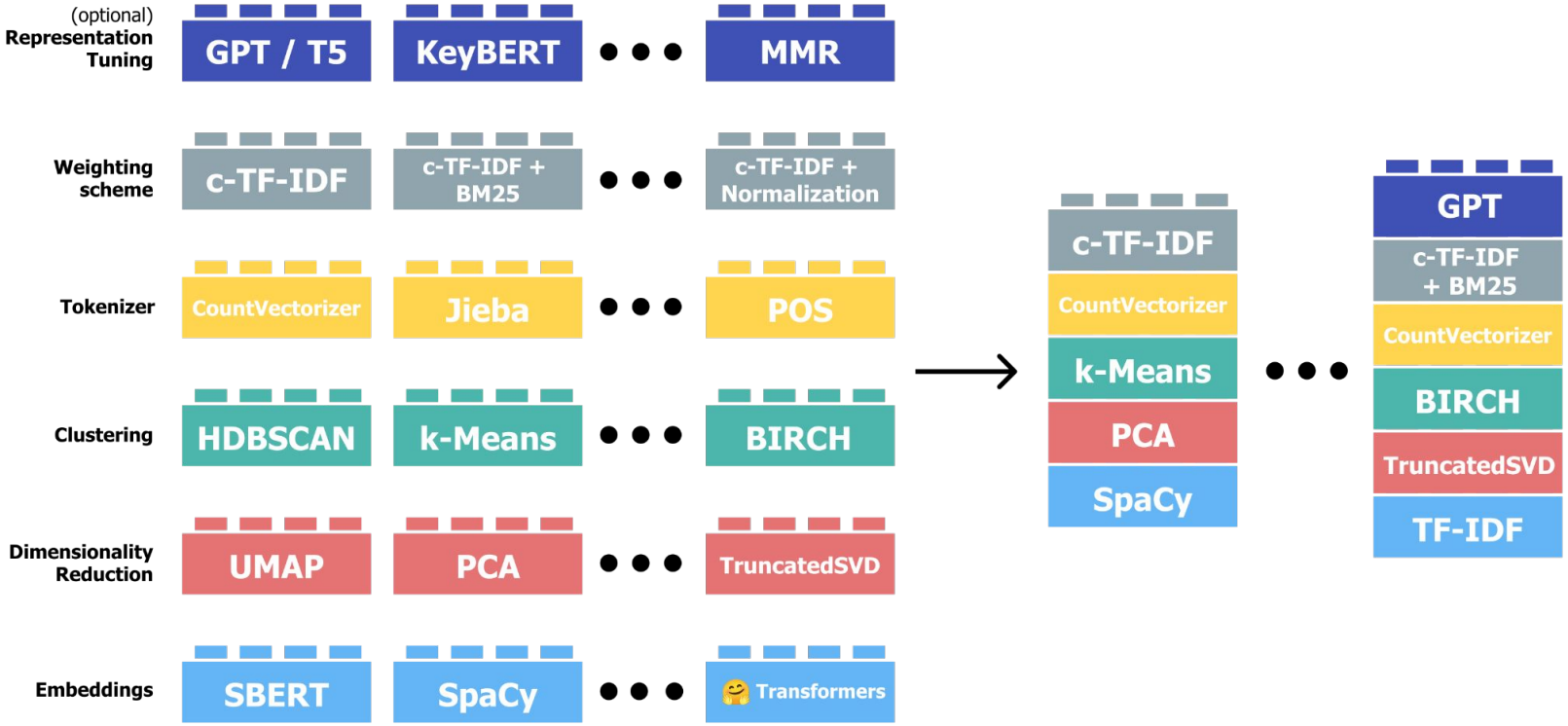


Arnaldo Santoro, PhD

✉ arnoud.data@gmail.com

arnaldo.santoro@unive.it

“standard BERTopic model”?



Representations

Keywords or Descriptions?

Keyword:

- first N highest scoring.
- c-TF-IDF: Standard
- MMR: important & diverse
- PoS: Part of Speech
- AI: KeyBERTInspired

Descriptive:

- AI-based:
  - Transformers
  - OpenAI
  - etc.
- DIY (“OI”-based)

Which Representation?

Keywords vs Descriptions

Keyword:

- Advantages:
  - objective
  - quantifiable
- Disadvantages:
  - difficult to understand

Descriptive:

- Advantages:
  - more understandable
- Disadvantages:
  - less objective
  - less quantifiable
  - access to resources

BoW: Limiti

n-grams: “Fatto Quotidiano”, “New York”.

Omografie: “pesca”, “accetta”, “capitano”, “principi”.

Enantiosemie: “storia”, “feriale”, “ospite”, “affittare”.

Posizione delle parole:

- “non voglio vino: sono astemio”
- “voglio vino: non sono astemio”

Preprocessing può modificare la semantica dei termini:

- “massa” -> “mass” <- “masso”

word2vec

- Word embedding.
- Introdotto nel 2013
- Allenato per prevedere parole collegate alla parola in input.
- Non usa tutto il contesto.
- Necessita tanti dati per funzionare meglio di BoW.
- Non gestisce Out Of Vocabulary (OOV).

Multi-Sense Skip-Gram

- Introdotto nel 2015
- Word embedding
- Usa parole vicine
- Assume 5 significati per ogni parola
- Non gestisce Out Of Vocabulary (OOV).

Static Embedding

Vantaggi:

- Ridotta dimensionalità.
- Le dimensioni sono concetti
- Può usare il contesto per disambiguare.

Limiti:

- Non usa tutto il contesto.
- Necessita tanti dati per funzionare meglio di BOW.
- Non gestisce Out Of Vocabulary (OOV).

Contextual Embedding

Vantaggi:

- Usa tutto il documento per creare embeddings.
- Gestisce Out Of Vocabulary (OOV).
- Modelli preallenati sono ampiamente disponibili.

Limiti:

- Si usano modelli preallenati.
- Non è veloce.

PCA?

Principal Component Analysis:

Semplice e veloce.

Molte assunzioni sui dati:

- Correlazioni lineari,
- Unicità dei “principal components”,
- Ortogonalità dei “principal components”.

t-SNE?

t-distributed Stochastic Neighbor Embedding:

Meglio di PCA.

MOLTO più lento di PCA.

Fa comunque troppe assunzioni sui dati.

UMAP 4/7

UMAP 5/7

c-TF-IDF: esempio

$$W_{t,c} = f_{t,c} \times log(1 + \frac{A}{f_t})$$

Per semplicità:

- logaritmo base 10
- $f_{t,c} = 1$
- A = 10.000, 100 topic con 100 documenti da 100 parole
- $f_t = 100$

Parola rara ($f_{t,c} = 1$) ma presente in ogni topic

Rappresentare Topic

$$W_{t,c} = f_{t,c} \times log(1 + \frac{A}{f_t})$$

$$W_{t,c} = 1 \times log_{10}(1 + \frac{10.000}{100})$$

$$= log_{10}(1 + 100)$$

$$\approx log(100) = 2$$

c-TF-IDF: esempio

$$W_{t,c} = f_{t,c} \times log(1 + \frac{A}{f_t})$$

Per semplicità:

- logaritmo base 10
- $f_{t,c} = 1$
- $A = 10.000$, 100 topic con 100 documenti da 100 parole
- $f_t = \mathbf{1}$

Parola rara ($f_{t,c} = 1$) ma presente in ogni topic

Rappresentare Topic

$$W_{t,c} = f_{t,c} \times log(1 + \frac{A}{f_t})$$

$$W_{t,c} = 1 \times log_{10}(1 + \frac{10.000}{1})$$

$$= log_{10}(1 + 10.000)$$

$$\approx log_{10}(10.000) = 4$$